

9주 차 보고서

(2026년 2월 15일 ~ 2026년 2월 28일)

작성자

박신우

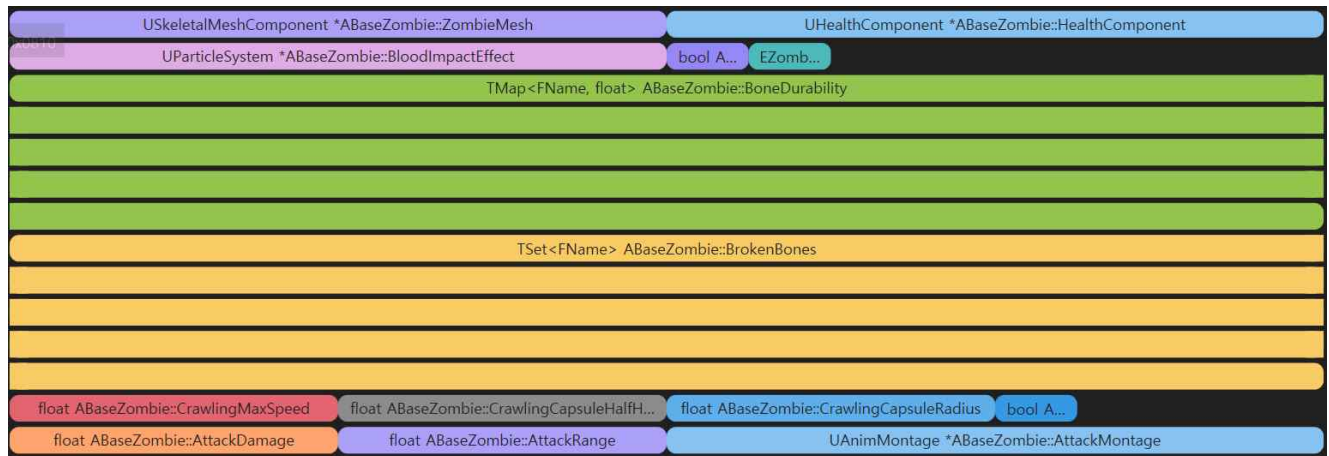
이번 주 한 일

2.18~2.21 클래스 구조 리팩터링

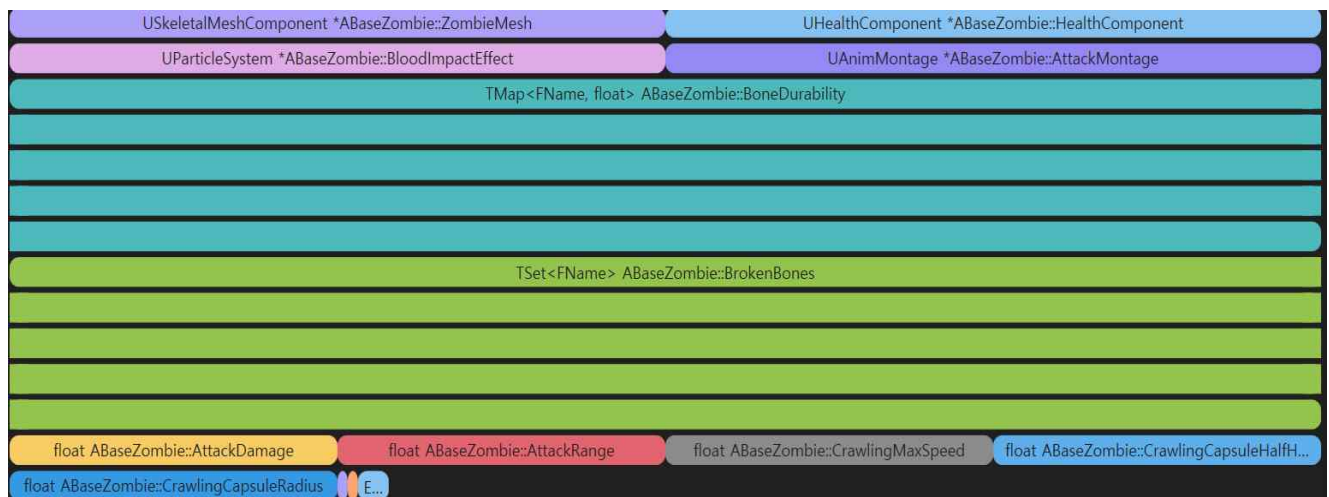
패딩을 입고 공부를 하러 도서관을 가고 있었는데,
문득 1학년때 c++시간에 메모리 패딩을 배웠던 것이 생각이 났다.
패딩 때문에 멤버 변수의 순서가 중요했던 것이 생각이 났는데,
“지금 내가 잘 지키고 있나?”라는 생각이 들었다.
그래서 코드를 확인해 보니 역시나 메모리 얼라인먼트가 개판이었다.

그래서 클래스들의 메모리 얼라인먼트를 고쳐주었다. 아래는 메모리 레이아웃 사진이다.

Before



After



이후에 캡슐화도 진행하여, 멤버 변수를 private 또는 protected로 변경하고,
외부접근은 Getter 와 Setter를 통해서 가능하도록 변경하였습니다.

이번 주 한 일

2.23 행동트리 공격 노드 생성

언리얼의 행동트리에서 호출하면 좀비가 공격하는 기능을 하는 노드를 만들었다.

좀비가 공격이 닿으면 플레이어의 체력이 닳도록 설정을 하였다.

기존 인간이 좀비를 공격할 때와 달리 굳이 HitResult 정보를 보내줄 필요가 없어, 데미지 값만 넘겨주는 ApplyDamage 함수를 HealthComponent에 만들어 주었다.

```
// HitResult 넘겨주지 않는 데미지 함수.  
void UHealthComponent::ApplyDamage(float Damage)  
{  
    if (Damage <= 0.f) return;  
  
    Health -= Damage;  
    Health = FMath::Max(Health, 0.f); // 체력 음수 방지  
  
    UE_LOG(LogTemp, Warning, TEXT("ApplyDamage: %f, Remaining Health: %f"), Damage, Health);  
  
    OnHealthChanged.Broadcast(Health, Damage);  
}
```

정상적으로 데미지가 들어오는 모습을 볼 수 있다.

```
LogTemp: Warning: Zombie BP_BaseZombie_C_2 Attack!  
LogTemp: Warning: Zombie BP_BaseZombie_C_2 Montage!  
LogTemp: Warning: ApplyDamage: 15.000000, Remaining Health: 70.000000  
LogTemp: Warning: Zombie dealt 15.000000 damage!  
LogTemp: Attack Montage Faded
```

2.24 행동트리 추격 노드 생성

좀비가 인간을 추격하는 기능도 노드로 만들어주었다.

기존의 기능을 하던 코드를 `UBTTask_ChaseHuman` 으로 옮겨주었는데,

아래 사진과 같이 한 번만 추격을 하고 도착을 하면 멈추는 문제가 생겼다.



이번 주 한 일

기존 코드에서는 ExcuteTask에서 플레이어에게 좀비가 접근하도록 하였다.
이렇게 하니, 1번만 좀비가 인간을 추적하고 멈추게 되었다.

그래서 TickTask를 만들어서 매 틱마다 좀비가 인간을 추적하도록 만들었다.

생성자에서 bNotifyTick을 참으로 해주고

```
UBTTask_ChaseHuman::UBTTask_ChaseHuman()
{
    NodeName = TEXT("Chase Human");
    // Blackboard 키 초기화
    TargetPlayerKey.SelectedKeyName = FName("TargetPlayer");

    // TickTask 활성화 (매 틱마다 실행)
    bNotifyTick = true;
}
```

ExcuteTask에서 InProgress를 반환해주면

```
// 이동 중이므로 In Progress 반환 (계속 실행)
return EBTNodeResult::InProgress;
```

행동트리가 “아, 이놈 아직 진행중이구나” 하고 TickTask를 호출한다.

TickTask에서 좀비를 추적하고 일정 거리에 도달하면 성공했다고 알려주도록 만들었다.

```
float Distance = FVector::Dist(ZombieCharacter->GetActorLocation(), TargetActor->GetActorLocation());

// 공격 범위에 도달하면 성공 반환 (공격으로 전환)
if (Distance <= StopDistance)
{
    UE_LOG(LogTemp, Warning, TEXT("In attack range! Distance: %f"), Distance);
    FinishLatentTask(OwnerComp, EBTNodeResult::Succeeded);
    return;
}

// 매 틱마다 플레이어 위치 업데이트
AIController->MoveToActor(TargetActor, StopDistance);
```

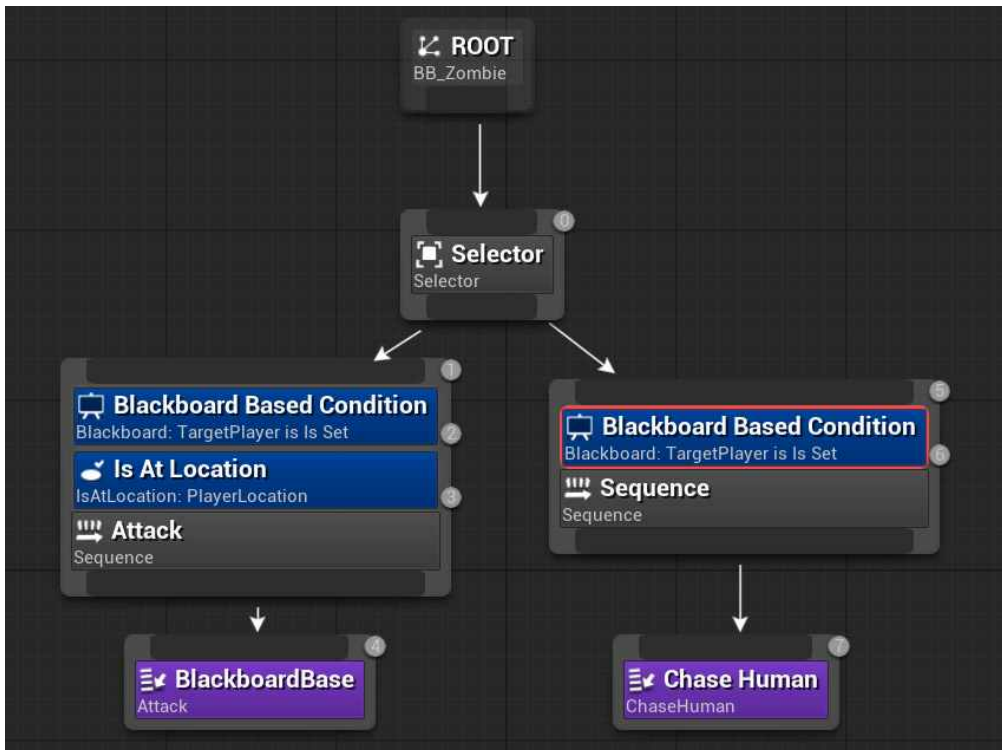
이후 정상적으로 추격하였다.

```
LogTemp: Chasing... Distance: 188.541931
LogTemp: Chasing... Distance: 182.791672
LogTemp: Chasing... Distance: 176.479309
LogTemp: Chasing... Distance: 169.604996
LogTemp: Chasing... Distance: 162.168610
LogTemp: Chasing... Distance: 154.168701
LogTemp: Warning: In attack range! Distance: 145.603729
```

이번 주 한 일

2.25 좀비 행동트리, 좀비 공격 애니메이션

다음과 같이 좀비 행동트리를 구성하였다.



Selector는 왼쪽의 노드부터 평가를 한다.

공격이 가능하면 공격을 하고 공격을 하는 조건이 만족하지않으면, 추격을 하도록 하였다.

플레이어가 감지되고, 공격 범위에 안에 있으면 공격을.

플레이어가 감지되고, 공격 조건이 만족하지않으면 추격을 하도록하였다.

좀비의 공격 애니메이션은 발로 공격하는 애니메이션을 믹사모에서 받아 적용하였다.

차후에 다른 공격 모션들도 넣어서 다양한 공격 모션을 섞어서 사용할 예정이다.

기존 좀비의 이동, 기어다니기 애니메이션은 애니메이션 블루프린트의 상태머신에 묶여있다.

이는 이동속도나 이동상태에 따라서 좀비의 애니메이션이 자연스럽게 변하기 위함이다.

하지만 공격과 같은 모션은 즉각적으로 나와야 해서, 몽타주를 사용해주었다.

걷는 도중에 공격을 해야하면, 즉각적으로 애니메이션이 나와 기존의 애니메이션을 덮어쓰도록 하였다.

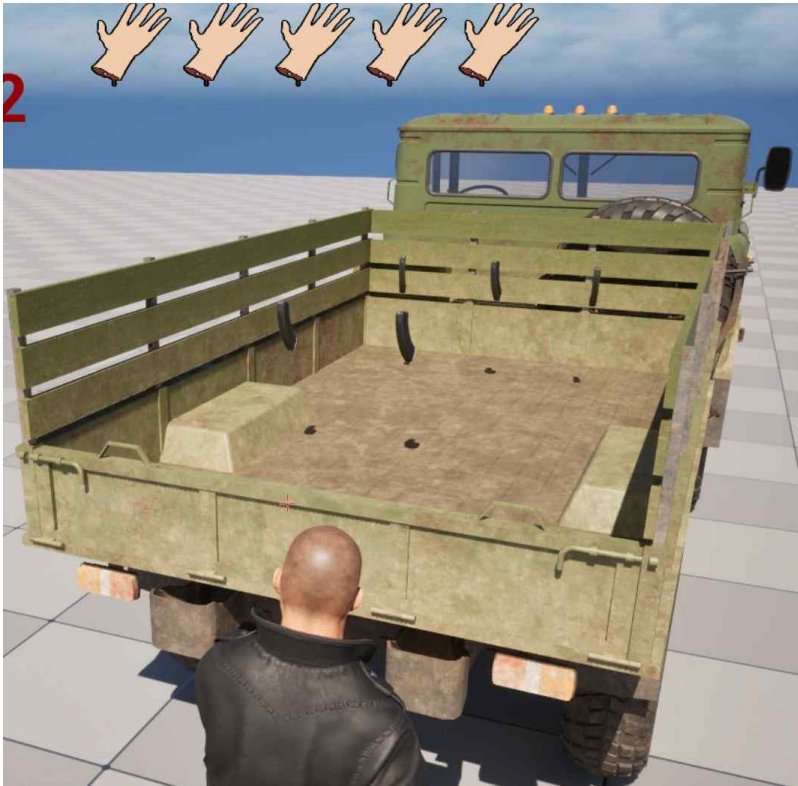


정상적으로 공격하는 모습을 확인할 수 있다.

이번 주 한 일

2.26 트럭 아이템 적재 모습 설정

기존에는 아래의 사진과 같이, 트럭에 아이템을 적재하면,
그 아이템의 메시가 일정거리 간격으로 동적으로 트럭위에 생성되도록 하였다.



하지만 동적으로 굳이 생성해야할 이유도 없고,
이쁘게 꾸미기도 힘들 것 같아서.
에디터에서 미리 이쁘게 메시를 쌓아두고 메시를 숨겨둔 뒤,
아이템이 적재되면 메시를 다시 보여주게 하는 방식으로 바꾸었다.

우선 종류별로 3개씩 컴포넌트로 만들어주었고.
에디터에서 위치와 매쉬를 설정할 수 있도록 해주었다

```
// AmmoSlots
for (int32 i = 0; i < 3; i++)
{
    FName SlotName = FName(*FString::Printf(TEXT("AmmoSlot_%d"), i));
    UStaticMeshComponent* NewSlot = CreateDefaultSubobject<UStaticMeshComponent>(SlotName);
    NewSlot->SetupAttachment(CargoOrigin);
    AmmoSlots.Add(NewSlot);
}

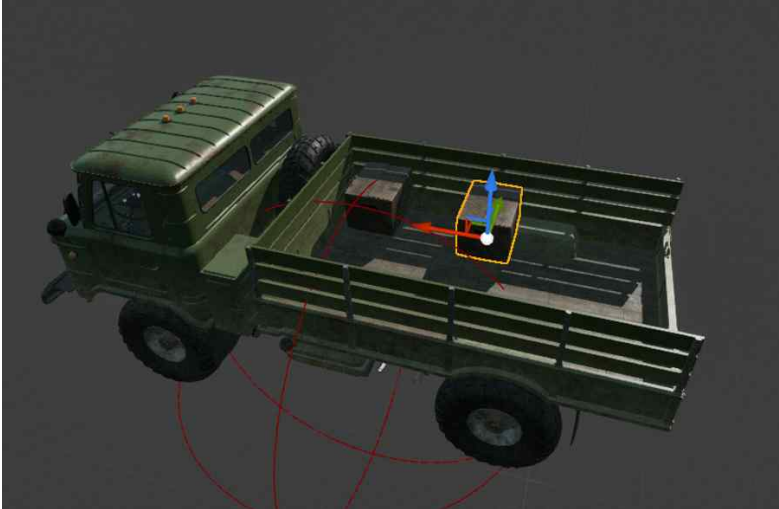
// FuelSlots
for (int32 i = 0; i < 3; i++)
{
    FName SlotName = FName(*FString::Printf(TEXT("FuelSlot_%d"), i));
    UStaticMeshComponent* NewSlot = CreateDefaultSubobject<UStaticMeshComponent>(SlotName);
    NewSlot->SetupAttachment(CargoOrigin);
    FuelSlots.Add(NewSlot);
}

// MedKitSlots
for (int32 i = 0; i < 3; i++)
{
    FName SlotName = FName(*FString::Printf(TEXT("MedKitSlot_%d"), i));
    UStaticMeshComponent* NewSlot = CreateDefaultSubobject<UStaticMeshComponent>(SlotName);
    NewSlot->SetupAttachment(CargoOrigin);
    MedKitSlots.Add(NewSlot);
}
```

Cargo Origin (CargoOrigin)		Edit in C++
	FuelSlot_1	Edit in C++
	FuelSlot_2	Edit in C++
	MedKitSlot_0	Edit in C++
	MedKitSlot_2	Edit in C++
	MedKitSlot_1	Edit in C++
	FuelSlot_0	Edit in C++
	AmmoSlot_2	Edit in C++
	AmmoSlot_1	Edit in C++
	AmmoSlot_0	Edit in C++

이번 주 한 일

아래와 같이 원하는 곳에 배치를 할 수 있고.



게임시작시엔 보이지않고, 아이템을 적재하면 모습이 보인다.

아이템 적재 전



적재 후



2.27 보고서 정리

다음 주 할 일

- 캐릭터 애니메이션
- 아이템 관련 회의

특이사항